

AI-Powered Incident Response Platform

Architecture, Design Decisions & Technical Reference

Multi-Agent • Kafka • MCP • pgvector • Multi-Tenant (Level 1) • OpenTelemetry

SDE-2 Portfolio Project

Table of Contents

Section	Page
1. Executive Summary	2
2. System Architecture (UML: Data Flow)	3
3. Agent Pipeline (UML: Sequence Diagram)	4
4. Kafka Topics & Configuration	5
5. MCP Tool Servers (UML: Architecture)	6
6. Multi-Tenancy — Level 1 Logical Isolation (UML)	7
7. pgvector & the Learning Loop	9
8. Data Models — Complete Reference	10
9. Database Schema	12
10. Infrastructure & Docker Compose	13
11. Technology Stack — With Justification	14
12. Build Phases — 10-Phase Roadmap	15
13. Interview Guide — Questions & Answers	16
14. Quick Reference	18

1. Executive Summary

What this platform does and why it matters

Mission: Reduce MTTR from ~45 minutes to ~8 minutes by autonomously triaging, diagnosing, and remediating production incidents using three specialized AI agents — without human intervention for low-risk fixes. Multi-tenant by design, observable end-to-end.

The Business Problem

Production incidents cost organizations an average of **\$5,600 per minute** in lost revenue, engineer time, and customer trust. The traditional on-call workflow is: alert fires → PagerDuty wakes an engineer → engineer spends 20 minutes understanding what is broken → 15 minutes diagnosing root cause → executes a fix. Total: 45 minutes. Most of that time is cognitive work — reading logs, querying metrics, looking up past incidents — that AI can do in seconds.

The Solution

An event-driven, multi-agent system where Kafka ingests alerts from Prometheus, Grafana, PagerDuty, and webhooks. Three specialized AI agents handle the full lifecycle:

- **Triage Agent** — classifies severity (P1–P4), retrieves the most relevant runbook using pgvector semantic search over past incidents and runbooks.
- **Diagnosis Agent** — calls MCP tool servers (Kubernetes, database, logs, metrics) to gather diagnostic evidence and identify the exact root cause.
- **Remediation Agent** — creates a remediation plan, routes each action through a risk classifier, requests human approval where needed, and executes the fix autonomously.

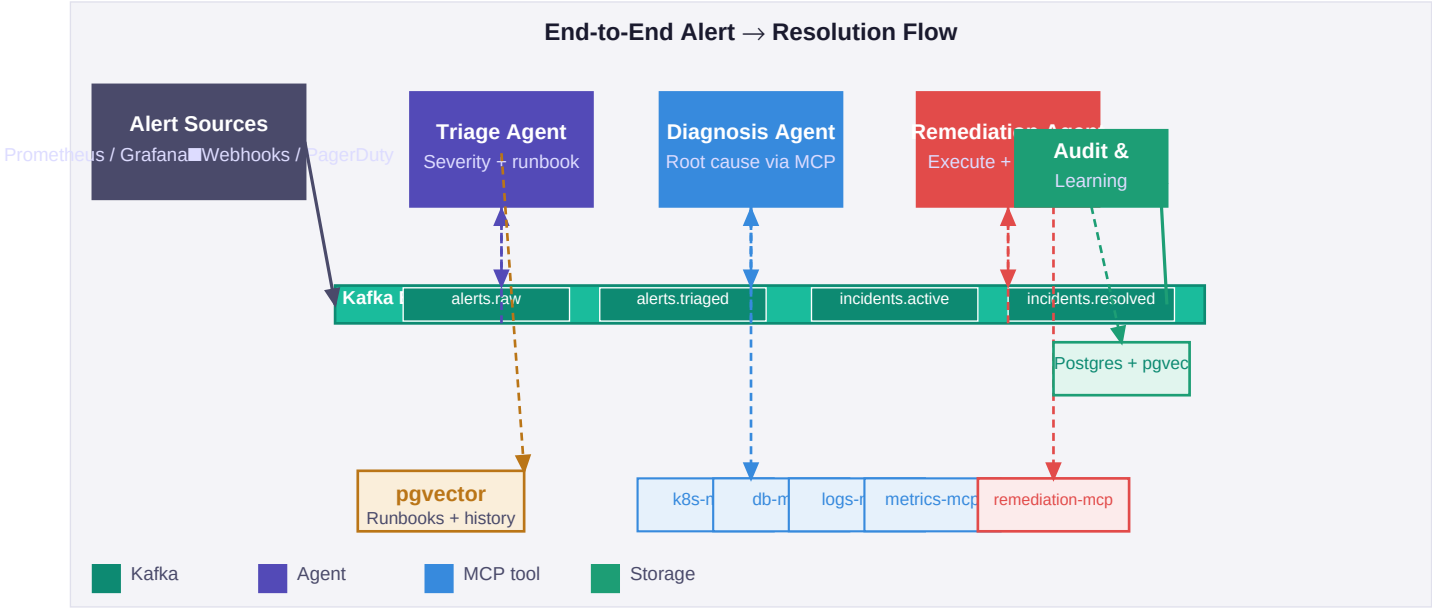
Primary KPI	MTTR — target reduction from ~45 min to ~8 min
AI model	claude-opus-4-5 via Anthropic SDK
Tool protocol	MCP (Model Context Protocol) — synchronous, not via Kafka
Event bus	Apache Kafka 3.5 — alert ingestion and agent handoffs only
Vector memory	PostgreSQL + pgvector — runbook and incident embeddings
Multi-tenancy	Level 1 — logical isolation, tenant_id in every layer
Language	Python 3.12, fully async (FastAPI, asyncpg, redis.asyncio)
Observability	OpenTelemetry — trace_id flows through Kafka headers + every service

2. System Architecture

Components, data flows, and design decisions

End-to-End Data Flow Diagram

Figure 1: Complete system data flow — alert sources through resolution and learning loop



Step	Component	Input	Output	Kafka Topic
1	Alert Ingestor	Prometheus / Grafana / Webhook	AlertEvent (normalized)	alerts.raw
2	Triage Agent	AlertEvent	IncidentContext + severity + runbook	alerts.triaged
3	Diagnosis Agent	IncidentContext	Root cause + evidence list	incidents.active
4	Remediation Agent	IncidentContext + RCA	Actions executed + resolution	incidents.resolved
5	Audit Consumer	All audit.events	Postgres row + Slack message	—
6	Learning Loop	Resolved incident	pgvector embedding upserted	—

Why Three Agents Instead of One

- **Permission isolation:** Triage and Diagnosis agents have zero write access. Only Remediation can call remediation-mcp — and only after human approval for medium/high risk.
- **Failure isolation:** If Diagnosis times out (Loki is down), the triage result is already persisted in Kafka. Retry from diagnosis step, not from scratch.
- **Auditability:** Each handoff is a discrete Kafka event — exact triage decision, exact RCA, exact remediation plan — as separate records, not buried in an LLM reasoning chain.

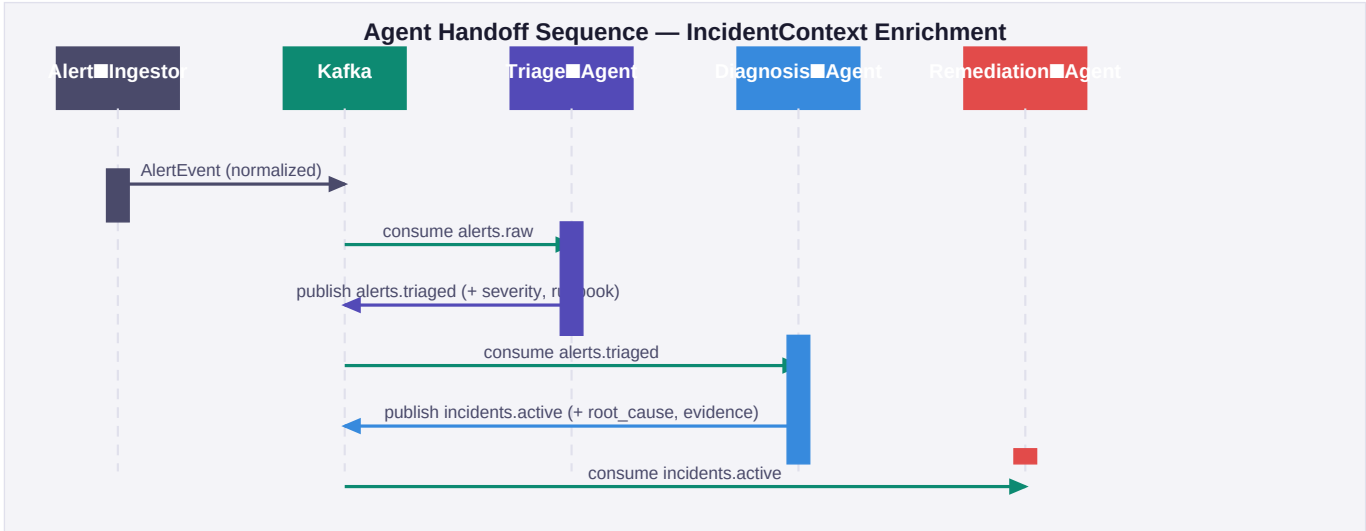
Kafka IS used for: Alert ingestion (bursty push events), agent pipeline handoffs (durable, replayable), audit event fan-out (multiple independent consumers).

Kafka is NOT used for: Agent→tool calls. Agents call MCP servers directly via HTTP because they need synchronous results to reason. Kafka would add 20–200ms latency with zero benefit.

3. Agent Pipeline — Sequence Diagram

IncidentContext is enriched as it passes through each agent

Figure 2: Agent handoff sequence — each agent enriches IncidentContext and publishes to next Kafka topic



The IncidentContext Object — Central Pipeline Memory

The IncidentContext is the single object that flows through the entire pipeline. Think of it as the incident's complete memory. It starts with just an AlertEvent and grows richer as each agent adds its findings:

After Triage Agent	+ severity (P1–P4), triage_summary, matched_runbook_id, similar_incident_ids, triage_confidence
After Diagnosis Agent	+ root_cause, affected_services, affected_components, evidence (list of MCP results), diagnosis_summary
After Remediation Agent	+ remediation_plan, human_approved, approved_by, resolved_at, resolution_summary

IncidentContext Status Lifecycle



Status transitions are strict — an agent can only move forward, never backward. If escalated, the incident is handed off to a human and the agents stop acting on it.

4. Kafka Topics & Configuration

Topic	Producer	Consumer(s)	Message Type	Retention
alerts.raw	alert-ingestor	triage-agent	AlertEvent (JSON)	7 days
alerts.triaged	triage-agent	diagnosis-agent	IncidentContext (JSON)	7 days
incidents.active	diagnosis-agent	remediation-agent	IncidentContext (JSON)	7 days
audit.events	all agents	audit-consumer (fan-out)	Audit event dict	30 days
incidents.resolved	remediation-agent	learning-loop	IncidentContext (JSON)	30 days

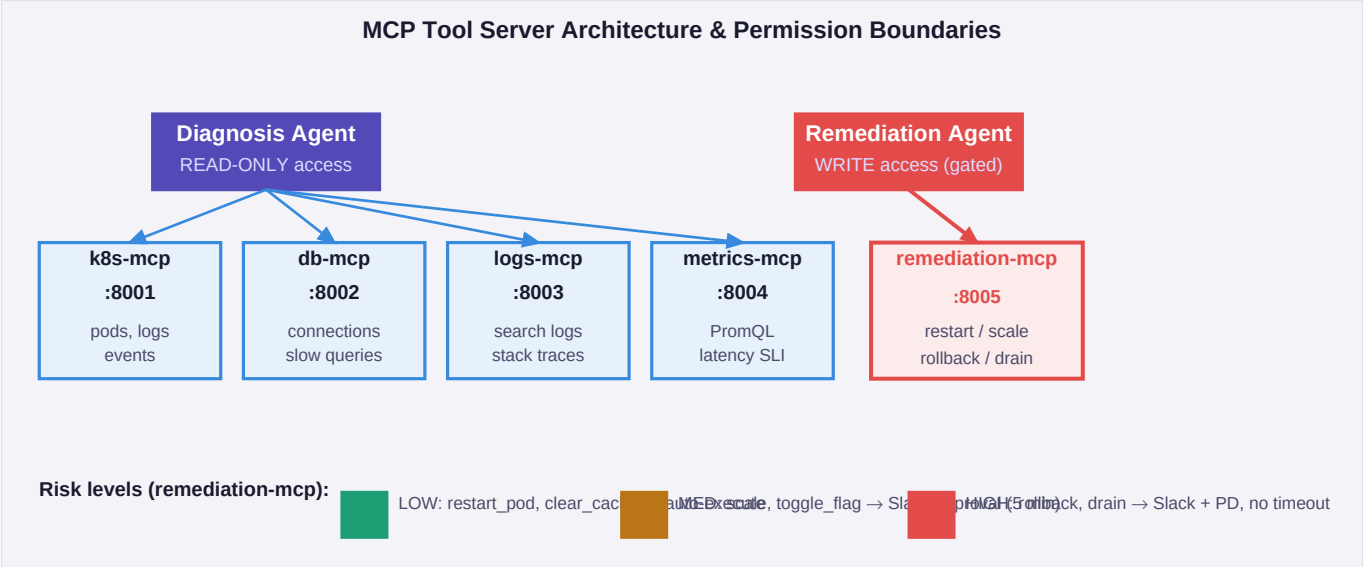
Critical Configuration Settings — Explained

- **num_partitions: 4** — four parallel consumers per topic. Run 4 triage-agent instances and Kafka distributes one partition to each. Horizontal scaling with zero config changes — just deploy more containers.
- **enable.auto.commit: false** — Kafka marks a message as processed (commits offset) only after the handler explicitly calls commit(). If the agent crashes mid-analysis, the message is redelivered. This is the most important reliability setting — auto-commit loses incidents.
- **max.poll.interval.ms: 300000** — 5-minute timeout before Kafka considers a consumer dead and redistributes its partitions. LLM + MCP tool calls can take 30–90 seconds; without this, Kafka evicts the consumer mid-processing and redelivers the message.
- **acks: all** — producer waits for all in-sync replicas to confirm before returning. In a 3-broker production cluster, this guarantees zero message loss even if one broker dies between the producer's send and the consumer's read.
- **log.retention.hours: 168** — messages retained for 7 days. If the diagnosis service is down for a day and comes back up, it replays all missed alerts. No incidents silently dropped.
- **consumer group isolation** — each service has its own group_id. Multiple consumers in the same group share partitions (load balancing). The audit consumer and learning loop both consume incidents.resolved in separate groups — both get every message independently.

5. MCP Tool Servers

Architecture, tools exposed, and risk classification

Figure 3: MCP tool server architecture — permission boundaries and risk levels



Server	Port	Agent Access	Access Type	Key Tools
k8s-mcp	8001	Diagnosis only	Read only	get_pod_status, get_pod_logs, get_recent_events, get_deployment_history, describe_service
db-mcp	8002	Diagnosis only	Read only	get_connection_count, get_slow_queries, get_replication_lag, check_table_bloat, get_lock_waits
logs-mcp	8003	Diagnosis only	Read only	search_logs, get_error_rate, get_stack_traces, correlate_by_trace_id
metrics-mcp	8004	Diagnosis only	Read only	query_prometheus, get_service_latency, get_error_rate_history, get_sli_status
remediation-mcp	8005	Remediation only	Write (risk-gated)	restart_pod (LOW), scale_deployment (MED), rollback_deployment (HIGH), drain_node (HIGH)

Human Approval Gate

Risk	Examples	Approval Workflow	Timeout
LOW	restart_pod, clear_cache	Auto-execute immediately, Slack notification after	None
MEDIUM	scale_deployment, toggle_feature_flag, restart_deployment	Post Slack request, wait for engineer to click Approve/Reject	5 min, then escalate
HIGH	rollback_deployment, drain_node	Post Slack + page PagerDuty. Mandatory human approval.	None — must respond

Slack approval format (MEDIUM/HIGH):

■ P1 — payment-service: connection pool exhausted

Root cause: 847 idle connections leaking from order-service

Fix: restart payment-service deployment [risk: MEDIUM | downtime: <10s]

Past incident: Oct 14 — same fix resolved in 4 min

[■ Approve] [■ Reject] [■ Show full RCA]

6. Multi-Tenancy — Level 1 Logical Isolation

Why Level 1, how tenant_id flows through every layer, and migration path

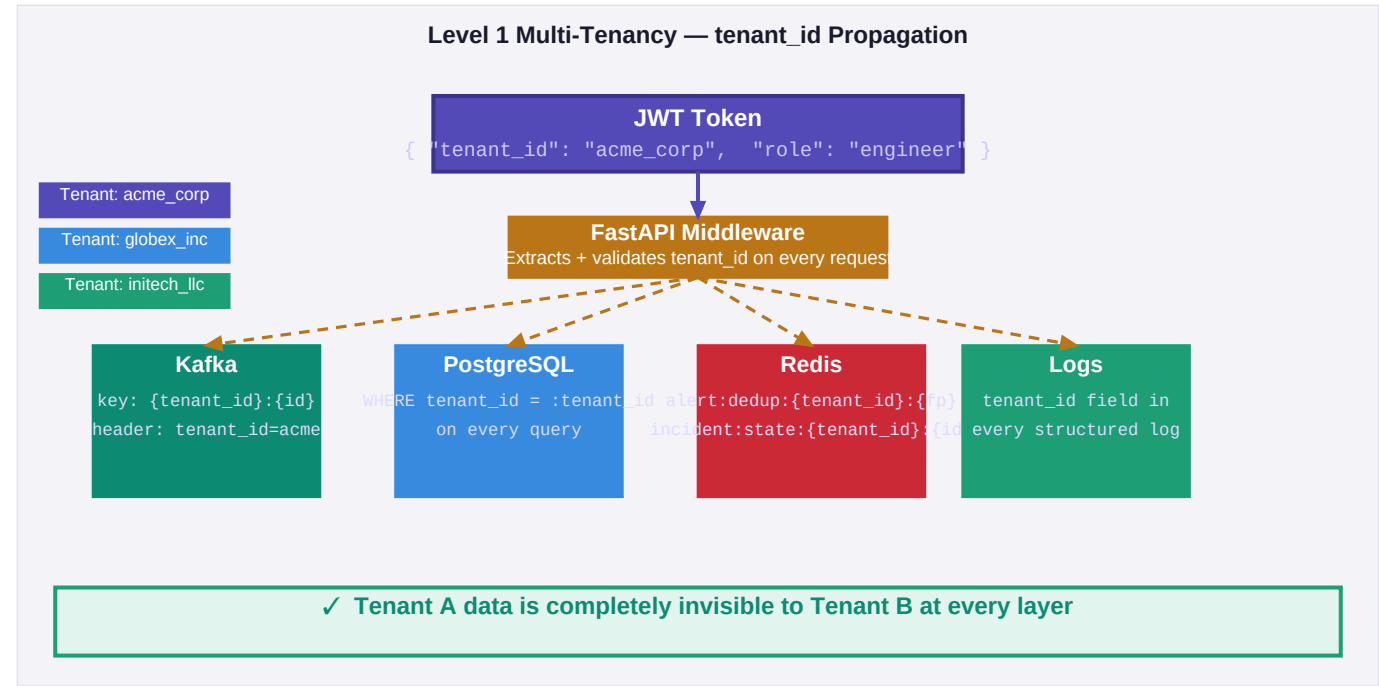
Architecture decision: Level 1 logical isolation — the standard for SaaS platforms with 10–10,000 tenants. Stripe, Slack, and Datadog all use this exact pattern. Level 2 (schema-per-tenant) and Level 3 (DB-per-tenant) add operational overhead without meaningful additional safety at this scale.

Level Comparison

Level	Isolation Method	Operational Cost	Best For	Used Here?
Level 1	tenant_id column + query filtering	Low — one schema, simple backups	10–10,000 tenants. Standard SaaS.	✓ Yes
Level 2	Separate Postgres schema per tenant	High — 50+ schemas, complex migrations	5–50 enterprise customers, data residency	—
Level 3	Separate DB instance per tenant	Very high — 100+ instances to manage	Enterprise-only (<20 customers)	—

tenant_id Propagation Diagram

Figure 4: tenant_id flows from JWT through every infrastructure layer



Implementation Detail — Each Layer

1. JWT — Single Source of Truth

Every user receives a JWT signed by the auth service. The token payload contains tenant_id and role. No other source of truth — if it's not in the JWT, it doesn't happen.

```
# JWT payload
{ "sub": "user_123", "tenant_id": "acme_corp",
  "role": "engineer", "exp": 1234567890 }
```

2. FastAPI Middleware — Enforcement at Request Boundary

Every service has a dependency function that decodes the JWT and injects `tenant_id`. No endpoint can be called without this — it's wired at the framework level, not per-route.

```
async def extract_tenant(request: Request) -> str:
    token = request.headers["Authorization"].split()[1]
    payload = jwt.decode(token, SECRET_KEY, algorithms=["HS256"])
    request.state.tenant_id = payload["tenant_id"]
    return payload["tenant_id"]

@app.post("/alerts/manual")
async def create_alert(alert: AlertEvent, tid: str = Depends(extract_tenant)):
    alert.tenant_id = tid # enforced at boundary, not trusted from client
```

3. Kafka — `tenant_id` in Key, Headers, and Payload

The Kafka message key is `{tenant_id}:{incident_id}`. This guarantees all messages for one tenant's incident go to the same partition (preserving order). The header carries `tenant_id` for fast filtering without deserializing the full payload.

```
producer.publish(
    topic="alerts.raw",
    value=incident.model_dump(),          # payload contains tenant_id
    key=f"{tenant_id}:{incident_id}",    # partition routing
    headers={"tenant_id": tenant_id, "trace_id": trace_id},
)
```

4. PostgreSQL — `WHERE tenant_id = :tenant_id` on Every Query

Every `SELECT`, `UPDATE`, and `DELETE` in the codebase includes a `tenant_id` filter. This is the most critical layer — a missing `WHERE` clause would return cross-tenant data. Comprehensive query tests catch this in CI before it reaches production.

```
async def get_incident(self, incident_id: str, tenant_id: str):
    result = await session.execute(text("""
        SELECT * FROM incidents
        WHERE incident_id = :incident_id
        AND tenant_id   = :tenant_id   -- always, always present
    """), {"incident_id": incident_id, "tenant_id": tenant_id})
```

5. Redis — `{tenant_id}`: Key Prefix

All Redis keys are prefixed with `tenant_id`. Without this, a cache entry for one tenant could be served to another, causing incorrect deduplication.

```
# Deduplication key – prevents duplicate incidents from flapping alerts
key = f"alert:dedup:{tenant_id}:{alert.fingerprint()}"

# Incident state key
key = f"incident:status:{tenant_id}:{incident_id}"

# Approval token key
key = f"approval:token:{tenant_id}:{request_id}"
```

Defense in Depth — 7 Enforcement Layers

Layer	How Enforced	Failure Mode if Missing
Middleware	JWT decoded, <code>tenant_id</code> extracted on every request	401 Unauthorized — request rejected

API input	alert.tenant_id = extracted_tenant_id (not from body)	Cannot spoof tenant via request body
Kafka key	{tenant_id}:{incident_id}	Messages mixed between tenants — hard to debug
Kafka headers	tenant_id header in every message	Consumer can't filter without full deserialization
DB queries	WHERE tenant_id = :tenant_id	Returns cross-tenant data silently
Redis keys	{tenant_id}: prefix on all keys	Cache collision — wrong tenant gets cached data
Structured logs	tenant_id field in every log line	Audit trail is unusable for investigation

Migration Path — From Level 1 to Level 2 (on-demand)

If a customer demands data residency (e.g., GDPR-enforced EU-only storage), migrate them to their own schema without impacting any other tenant:

```
-- 1. Create isolated schema
CREATE SCHEMA tenant_acme;

-- 2. Copy and migrate data
CREATE TABLE tenant_acme.incidents AS
  SELECT * FROM incidents WHERE tenant_id = 'acme_corp';

-- 3. Route acme's JWT to the new schema in settings
TENANT_SCHEMA_MAP = { 'acme_corp': 'tenant_acme', 'default': 'public' }

-- 4. Clean up public schema
DELETE FROM incidents WHERE tenant_id = 'acme_corp';
-- All other tenants: zero impact, zero downtime
```

7. pgvector & the Learning Loop

How the system gets smarter with every resolved incident

Why pgvector Instead of a Dedicated Vector Database

The incidents and runbooks already live in Postgres. Using Pinecone, Weaviate, or Chroma would require two network round trips per triage: one to the vector DB for similarity search, one to Postgres for the actual incident metadata — then joining in application code. With pgvector, a single SQL query returns both. At the scale of thousands of incidents and hundreds of runbooks, the HNSW index is fast enough — typically under 10ms.

How Similarity Search Works — Step by Step

- **Step 1:** Triage Agent receives AlertEvent: name='HighErrorRate', service='payment-service', summary='Error rate above 10% for 5 minutes'.
- **Step 2:** Embed the combined text using an embedding model → 1536-dimensional float vector.
- **Step 3:** Run pgvector cosine similarity query: `SELECT ... FROM runbooks ORDER BY embedding <=> :vec LIMIT 3`.
- **Step 4:** Same query against incidents `WHERE status='resolved'` → top-3 similar past incidents.
- **Step 5:** Inject results into Claude's context: 'Here are the 3 most relevant runbooks and 3 most similar past incidents for this alert.'
- **Step 6:** Claude's triage answer already contains: 'We saw this in October — root cause was connection pool exhaustion, fixed by restarting app-server in 4 minutes.'

The HNSW Index

Without an index, pgvector does exact nearest-neighbor search: $O(n)$. At 10,000 incidents this takes 500ms+. The HNSW (Hierarchical Navigable Small World) index builds a multilayer graph where each vector links to its nearest neighbors, reducing search to $O(\log n)$ — typically 5–15ms at the same scale:

```
CREATE INDEX idx_incidents_embedding ON incidents
  USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 128);

-- m=16: each node links to 16 neighbors (higher = more accurate, more memory)
-- ef_construction=128: build-time quality (higher = better index, slower build)
-- vector_cosine_ops: cosine distance – scale-invariant, best for text embeddings
```

Learning Loop — Postmortem to Embedding

After every resolved incident, the learning-loop Kafka consumer is triggered:

- **Step 1:** Consume incidents.resolved Kafka message (IncidentContext with resolved_at populated).
- **Step 2:** Generate embedding of: '{alert_name} {service} {root_cause} {resolution_summary}'.
- **Step 3:** `UPDATE incidents SET embedding = :vector WHERE incident_id = :id AND tenant_id = :tid`.
- **Step 4:** The HNSW index is updated automatically by Postgres on every INSERT/UPDATE.
- **Step 5:** Next similar alert → pgvector finds this incident → triage agent has historical context.

8. Data Models — Complete Reference

AlertEvent

The canonical normalized alert. Every source is normalized into this shape at the ingestor boundary. Downstream services never deal with source-specific formats.

Field	Type	Description
alert_id	str (uuid4)	Auto-generated unique identifier
tenant_id	str	Tenant that owns this alert — set by middleware, never trusted from client
source	AlertSource enum	prometheus grafana pagerduty datadog webhook manual
status	AlertStatus enum	firing resolved silenced
severity	Severity None	None until triage agent sets it — P1/P2/P3/P4
name	str	Alert name: 'HighErrorRate', 'PodCrashLooping'
service	str	Affected service: 'payment-service', 'order-api'
environment	str	Default 'production'
labels	dict[str,str]	{'env':'prod','team':'payments'} — from Prometheus
annotations	AlertAnnotation	summary, description, runbook_url
raw_payload	dict	Original source JSON — preserved for debugging
fired_at	datetime	When alert condition triggered at source
trace_id	str None	OpenTelemetry trace ID — added in Phase 8

Key method: **fingerprint()** → returns '{name}:{service}:{environment}'. Used by Redis deduplication to prevent multiple identical incidents when an alert flaps.

IncidentContext

The central object flowing through the pipeline. Every agent reads it, enriches it, and publishes the updated version to the next Kafka topic. Fields are grouped by which agent sets them.

Field	Set By	Type	Description
incident_id	auto	str (uuid4)	Primary key throughout system
tenant_id	middleware	str	Forwarded from AlertEvent, never changed
status	each agent	enum	triaging → diagnosing → remediating → resolved
alert	ingestor	AlertEvent	Original alert — immutable after creation
severity	triage	Severity	P1/P2/P3/P4
triage_summary	triage	str	Human-readable: what is happening
matched_runbook_id	triage	str None	Best matching runbook from pgvector

similar_incident_ids	triage	list[str]	Past incident IDs used as context
triage_confidence	triage	float	0.0–1.0 — Claude's certainty in classification
root_cause	diagnosis	str	Root cause identified by diagnosis agent
affected_services	diagnosis	list[str]	All services impacted (may be > 1)
evidence	diagnosis	list[Evidence]	MCP tool results collected during diagnosis
remediation_plan	remediation	list[Action]	Full action plan — written before execution
human_approved	remediation	bool	True after Slack approval received
approved_by	remediation	str None	Slack user ID of approving engineer
resolved_at	remediation	datetime	Timestamp for MTTR calculation
trace_id	ingestor	str None	OTel trace ID — flows through all stages

Evidence & Action

Evidence.source	Which MCP server produced this — 'k8s-mcp', 'logs-mcp', 'db-mcp'
Evidence.tool_name	Which tool was called — 'get_pod_logs', 'search_logs'
Evidence.content	Actual data returned (log lines, metric values, k8s events)
Evidence.relevance	Agent's explanation of why this evidence matters for root cause
Action.tool_fn	Tool function to call — 'restart_pod', 'scale_deployment'
Action.parameters	Arguments — {'namespace':'prod','deployment':'payment-service'}
Action.risk_level	LOW MEDIUM HIGH — determines approval workflow
Action.reasoning	Why agent chose this action — preserved for audit
Action.executed	False until after approval + execution

9. Database Schema

The init.sql runs automatically on first Postgres container start via docker-entrypoint-initdb.d. It enables pgvector, creates all tables with proper indexes, and seeds three starter runbooks.

Table	Key Columns	Purpose
incidents	incident_id, tenant_id, status, service, severity, root_cause, mtrr_seconds, raw_context JSONB, embedding vector(1536)	One row per incident. raw_context stores full IncidentContext as JSONB. embedding populated after resolution.
runbooks	runbook_id, tenant_id, title, description, services TEXT[], steps JSONB, embedding vector(1536)	Manually written runbooks. Steps as JSONB for flexibility. Embedding for pgvector search.
audit_events	event_id, incident_id (FK), agent, action, details JSONB, trace_id	Immutable audit log. One row per agent action. Used for compliance and postmortem analysis.
approval_requests	request_id, incident_id (FK), action JSONB, risk_level, slack_ts, status, expires_at	Tracks pending approvals. slack_ts allows live-updating the Slack message when engineer responds.

pgvector Index Configuration

```
-- HNSW index for fast similarity search (both tables)
CREATE INDEX idx_incidents_embedding ON incidents
  USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 128);

CREATE INDEX idx_runbooks_embedding ON runbooks
  USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 128);

-- Standard indexes for common query patterns
CREATE INDEX idx_incidents_status ON incidents(status);
CREATE INDEX idx_incidents_tenant ON incidents(tenant_id);
CREATE INDEX idx_incidents_service ON incidents(service);
CREATE INDEX idx_audit_incident ON audit_events(incident_id);
CREATE INDEX idx_approvals_status ON approval_requests(status);
```

Cosine Similarity Query Pattern

```
SELECT
  runbook_id, title, description, steps,
  1 - (embedding <=> CAST(:vec AS vector)) AS similarity
FROM runbooks
WHERE tenant_id = :tenant_id
  AND (service = ANY(services) OR :service = ANY(services))
ORDER BY embedding <=> CAST(:vec AS vector) -- <=> is cosine distance
LIMIT 3;

-- 1 - distance = similarity score (0=no match, 1=identical)
-- Cosine is scale-invariant: short alert matches long runbook if semantically similar
```

10. Infrastructure & Docker Compose

Service	Image	Port(s)	Purpose	Health Check
zookeeper	cp-zookeeper:7.5.3	2181	Kafka control plane	echo ruok nc
kafka	cp-kafka:7.5.3	9092, 29092	Message broker	kafka-broker-api-versions
schema-registry	cp-schema-registry:7.5.3	8081	Message schema validation	HTTP /subjects
kafka-ui	provectuslabs/kafka-ui	8080	Visual broker management	HTTP /
redis	redis:7.2-alpine	6379	Dedup, state, approval tokens	redis-cli ping
postgres	pgvector/pgvector:pg16	5432	Incidents, runbooks, pgvector	pg_isready
prometheus	prom/prometheus:v2.48.0	9090	Metrics scraping	HTTP /-/healthy
grafana	grafana/grafana:10.2.0	3000	Dashboards (MTTR, incidents)	HTTP /api/health

Kafka Dual Listener — Why Two Ports

Kafka advertises the address clients should use to connect. Services inside Docker use **kafka:9092** (Docker internal network). Scripts on your local machine use **localhost:29092** (mapped out of Docker). Same broker, two listeners, two audiences. Without this, local test scripts can't reach Kafka and services inside Docker use the wrong address.

Docker Healthchecks — Why They Matter

Every service has a healthcheck block and depends_on: condition: service_healthy. Without this, Kafka starts before Zookeeper is ready and crashes. Postgres accepts connections before init.sql has finished running. The healthchecks guarantee startup order without arbitrary sleep commands.

11. Technology Stack — With Justification

Every technology chosen to solve a specific problem

Component	Technology	Why This, Not the Alternative
AI agents	Python + anthropic SDK	Native MCP support; tool-use and streaming first-class
MCP servers	fastmcp library	10-line MCP server; Claude discovers tools automatically
Message broker	Apache Kafka 3.5	Alert bursts; durable handoffs; fan-out consumers — Celery can't do all three
Vector memory	PostgreSQL + pgvector	No extra service; join vector + relational in one query — Pinecone requires two network calls
Session state	Redis (async)	Sub-ms dedup; TTL-based approval tokens — SQLite is sync only
Persistence	PostgreSQL 16	JSONB for IncidentContext; pgvector for embeddings; ACID — MongoDB loses structured queries
HTTP client	httpx (async)	Native async; connection pooling — requests is sync only
Notifications	Slack Bolt SDK	Interactive approvals (buttons); event handling — plain webhooks are fire-and-forget
Observability	OpenTelemetry + Grafana Tempo	Vendor-neutral; trace_id propagates through Kafka headers
Containers (dev)	Docker Compose	One command; healthchecks; reproducible — Makefile scripts are fragile
Containers (prod)	Kubernetes + HPA	Auto-scale by Kafka consumer lag metric — Docker Swarm has limited autoscaling
CI/CD	GitHub Actions	Native to GitHub; matrix builds; zero infra to manage
Logging	structlog (JSON)	Searchable fields; trace_id injection — stdlib logging is unstructured

12. Build Phases — 10-Phase Roadmap

~25 days total at 2 hours/day — each phase is self-contained and testable

Phase	What Gets Built	Key Concepts Learned	Days
1	Monorepo scaffold, Docker Compose (Kafka+Postgres+Redis), shared Pydantic models, Postgres init.sql with pgvector	Monorepo design, Docker healthchecks, Pydantic BaseSettings, pgvector schema, HNSW indexes	2
2	Alert ingestor — FastAPI endpoints for Prometheus Alertmanager, Grafana, webhooks; normalize to AlertEvent	Alert normalization, Redis fingerprint dedup, Kafka producer, manual offset commit	1
3	Triage Agent — Claude with pgvector runbook retrieval	Embedding generation, cosine similarity, HNSW index, agentic calls, confidence scoring	3
4	Four diagnostic MCP servers — k8s-mcp, db-mcp, logs-mcp, metrics-mcp	fastmcp library, Kubernetes Python SDK, asyncpg, httpx, PromQL queries	4
5	Diagnosis Agent — multi-turn MCP tool calling loop with evidence collection	Agentic loops, tool-use content blocks, evidence accumulation, context management	3
6	Remediation Agent — plan, risk classify, human approval gate via Slack	Risk classifier, Slack Bolt SDK, approval token storage in Redis, remediation-mcp	3
7	Audit consumer — Postgres persistence + Slack incident notifications	Fan-out consumer groups, async SQLAlchemy upsert, Slack message formatting	2
8	Learning loop — resolved incident postmortem to pgvector embedding	Embedding pipeline, pgvector upsert, async batch processing, HNSW index update	2
9	OpenTelemetry — trace_id propagation through Kafka headers + all services	OTel SDK, context propagation, Kafka header injection, Grafana Tempo	2
10	Dashboard (WebSocket live view), CI/CD (GitHub Actions), Kubernetes manifests	FastAPI WebSocket, HPA autoscaling by Kafka lag, GitHub Actions matrix, K8s secrets	3

Total: ~25 days @ 2 hours/day. A production-grade, SDE-2 portfolio project.

13. Interview Guide

How to present this project — 90-second opener and answers for every expected question

90-Second Opening — Memorize This

"Production incidents cost an average of \$5,600 per minute. I built a platform that reduces MTTR from 45 minutes to ~8 minutes by having three specialized AI agents autonomously handle the full incident lifecycle. The interesting engineering challenge was deciding what to automate and what to keep human-controlled — low-risk fixes execute automatically, high-risk actions always require explicit human approval via Slack, no matter how confident the agent is. The system learns from every resolved incident: embeddings of past root causes and resolutions are stored in pgvector, so the triage agent's first response to a new alert already includes relevant historical context. It's multi-tenant from day one — tenant_id flows through every layer: JWT, Kafka partition keys, every database query, and Redis key prefixes."

Expected Questions — Full Answers

Q1: Why Kafka? Couldn't you use a simple task queue like Celery or SQS?

A: Kafka gives you three things Celery and SQS can't: durable, replayable message history so if the diagnosis service is down for an hour, no alerts are lost when it restarts; true fan-out consumers — the audit consumer, learning loop, and dashboard all independently consume the same audit.events topic with zero coupling; and built-in backpressure so if all triage-agent instances are busy, Kafka queues alerts rather than dropping them. Critically, I did NOT use Kafka for agent-to-tool communication — that stays direct HTTP via MCP because the agent needs synchronous results to reason. Kafka for tool calls would add latency with zero benefit.

Q2: Why three agents instead of one big agent?

A: Permission isolation, primarily. The diagnosis agent only has read access to Kubernetes, logs, and metrics. The remediation agent has write access — but only after human approval for medium and high risk actions. If I merged them into one agent, I'd need to give it write access during the diagnosis phase, violating least privilege. Secondly, failure isolation: if diagnosis times out because Loki is down, the triage result is already saved in Kafka. The incident can be retried at diagnosis without re-running triage. In a single-agent system, everything restarts.

Q3: How does the agent avoid making things worse with autonomous remediation?

A: Three layers. First, the risk classifier: every action is LOW, MEDIUM, or HIGH based on type and blast radius. Second, the approval gate: LOW auto-executes with post-hoc notification, MEDIUM requires Slack approval within 5 minutes or it escalates, HIGH requires explicit approval with no timeout. Third, the audit trail: every planned and executed action is recorded in Postgres and Kafka before execution begins, so we can always reconstruct exactly what the agent did and why.

Q4: How is multi-tenancy implemented?

A: Level 1 — logical isolation, standard for SaaS. Every JWT carries tenant_id, extracted by middleware at the request boundary. It flows through Kafka partition keys and message headers, every Postgres query has WHERE tenant_id = ?, and Redis keys are prefixed with tenant_id. Defense in depth at 7 layers. If a customer needs data residency compliance, I migrate them to their own Postgres schema on-demand without touching any other tenant. But at 10–10,000 customers, Level 1 is the right choice — it's how Stripe, Slack, and Datadog work.

Q5: Why pgvector instead of Pinecone or a dedicated vector database?

A: Incidents and runbooks already live in Postgres. With pgvector, one SQL query returns the similarity score AND all incident metadata. With Pinecone, I'd make a vector search call, get back IDs and scores, make a Postgres call to fetch the data, then join in application code — two network round trips and more code. With the HNSW index, pgvector runs similarity queries in under 10ms at thousands of incidents. No additional service to operate, monitor, or pay for.

Q6: How does the system improve over time without retraining?

A: After every resolved incident, the learning-loop service generates an embedding of the combined alert name, root cause, and resolution summary, then stores it in the incidents table. The next time a similar alert fires, the triage agent retrieves the top-3 most similar past incidents via pgvector and injects them into Claude's context. Claude then says 'we saw this in October — root cause was connection pool exhaustion, fixed by restarting the app server, 4 minutes.' The memory is the database — no fine-tuning, no RAG pipeline, no separate vector service.

14. Quick Reference

Commands, ports, and URLs for daily development

Start the Full Stack

```
cd infra && docker compose up -d      # Start all infrastructure
docker compose ps                      # Verify all services are 'healthy'
open http://localhost:8080             # Kafka UI – see topics, messages, consumer lag
open http://localhost:3000             # Grafana – MTTR dashboard (admin/admin)
open http://localhost:9090             # Prometheus – raw metrics
```

Send a Test Alert

```
curl -X POST http://localhost:8000/alerts/manual \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $JWT_TOKEN" \
-d '{
  "name": "HighErrorRate",
  "service": "payment-service",
  "summary": "Error rate above 10% for 5 minutes",
  "environment": "production"
}'
```

All Service Ports

Service	Port	URL / Access
alert-ingestor	8000	http://localhost:8000/docs — Swagger UI
k8s-mcp	8001	http://localhost:8001/mcp — MCP endpoint
db-mcp	8002	http://localhost:8002/mcp
logs-mcp	8003	http://localhost:8003/mcp
metrics-mcp	8004	http://localhost:8004/mcp
remediation-mcp	8005	http://localhost:8005/mcp
dashboard	8006	http://localhost:8006 — WebSocket live view
Kafka UI	8080	http://localhost:8080 — visual broker management
Schema Registry	8081	http://localhost:8081
Prometheus	9090	http://localhost:9090
Grafana	3000	http://localhost:3000 — admin/admin
Redis	6379	redis-cli -h localhost -p 6379
Postgres	5432	psql -U agent_user -d incident_db -h localhost
Zookeeper	2181	docker exec zookeeper zkCli.sh

Useful Kafka Commands

```
# List all topics
docker exec kafka kafka-topics --bootstrap-server localhost:9092 --list

# Describe a topic (partitions, offsets)
docker exec kafka kafka-topics --bootstrap-server localhost:9092 --describe --topic alerts.raw

# Consume a topic from the beginning
docker exec kafka kafka-console-consumer --bootstrap-server localhost:9092 \
  --topic alerts.raw --from-beginning

# Check consumer group lag (critical for production monitoring)
docker exec kafka kafka-consumer-groups --bootstrap-server localhost:9092 \
  --describe --group triage-agent-group
```